

HAI911I – Développement d'applications Interactives :

Texture 3D

Noura Faraj : noura.faraj@umontpellier.fr

Sylvain Leclerc : sylvain.leclerc@lirmm.fr

Objectif :

Le but de ce TP est d'utiliser une texture 3D afin de visualiser l'intérieur d'une grille de voxel.

Nous allons pour ça utiliser la librairie libQGLViewer permettant de faire des applications graphiques avec Qt.

Décompresser la base de code « Code Texture 3D »

Dans le dossier « Texture 3D »

```
qmake  
make -j
```

Notez que le qmake permet de générer le Makefile qui sera utilisé pour la compilation. Ouvrez le fichier .pro avec QtCreator, configurez le projet pour que le dossier de compilation soit le dossier Texture3D.

Ou utilisez un bon IDE et compilez en ligne de commande.

Familiarisez-vous avec le code.

La classe Window permet de définir la fenêtre de l'application, un dockWidget permet de définir la barre amovible d'outils sur la droite et le TextureViewer (dérivant de QGLViewer) définit le widget central de la fenêtre.

Nous pouvons charger une image à l'aide du menu File > Load 3D Image, vous voyez apparaître la boîte englobante de la grille de voxel sélectionnée dans la fenêtre graphique. Ici nous nous intéressons aux grilles de voxel contenant des unsigned char. Lors de l'ouverture de l'image une couleur est affectée à chacune des valeurs présentes dans la grille.

Nous allons plaquer une texture 3D à la boîte englobante de l'image 3D représentée (notez que ses coordonnées maximales sont (xMax, yMax, zMax) = (dx*nx, dy*ny, dz*nz)) avec dx,dy,dz les dimensions d'un voxel de l'image et nx,ny,nz le nombre d'éléments dans chacune des dimensions).

Question 1 : Connexion des signaux

- Connecter les signaux **envoyés par le widget Texture3dDockWidget** lors d'un changement de valeurs des sliders des trois axes (xValueChanged, yValueChanged, zValueChanged), aux fonctions de l'objet viewer, permettant de mettre à jour les positions des plans de coupe alignés sur les axes (setXCut, setYCut, setZCut).
- Connecter également les signaux permettant de changer la direction de visibilité (signal de clique de xInvert avec invertXCut etc).

- Connecter les radios buttons permettant d'afficher ou non les plans de coupe. (xDisplay avec setXCutDisplay etc).

Question 2 : Création et envoi de la texture

Dans la classe Texture, créer et envoyer la texture 3D contenant les valeurs de la grille de voxel sur le GPU :

- Dans la fonction build de la classe Texture remplir les valeurs RGBA des Texels avec les couleurs définies dans labelsToColor.
- Compléter la fonction initTexture() pour définir les paramètres de la texture (interpolation linéaire, plus proche voisin etc.)

Question 3 : Affichage de la texture sur les plans de coupe

- Dans la méthode draw de Texture, envoyer les Uniforms xMax, yMax, zMax et calculer les coordonnées de texture dans le vertex shader (note on utilisera les positions 3D normalisées).
- Activer et binder la texture 3D.
- Compléter le Vertex shader pour calculer les coordonnées de texture. (note : Lorsque vous modifiez uniquement les shaders, vous pouvez utiliser le bouton recompile shaders de l'interface QT, sans relancer votre programme)
- Dans le Fragment Shader, assigner la couleur de la texture 3D au fragment en utilisant les coordonnées de texture calculées précédemment.
- Envoyer les Uniforms permettant de définir les plans de coupes alignés sur les axes : xCutPosition, yCutPosition, zCutPosition définissant les positions des plans et xCutDirection, yCutDirection, zCutDirection définissant le coté du plan le volume sera visible. Notez que ceux-ci vont être mis à jour grâce aux sliders.
- Dessiner des quads représentant les plans de coupe grâce à la fonction drawCutPlanes() et vérifiez le bon fonctionnement de votre programme.
- Complétez ensuite la fonction ComputeVisibility() du Fragment Shader. Pour cela, vérifiez que le fragment courant se trouve du côté visible de chacun des plans (piste : utiliser le produit scalaire avec un point des plans et les axes principaux en utilisant les cutDirection). Dessinez en plus la boîte englobante, quel résultat obtenez-vous ?

Question 4 : Affichage de la texture sur un maillage quelconque

- En vous inspirant de l'action Load 3D Image, dans le menu File ajouter une option pour ouvrir un maillage 3D au format off, vous pouvez utiliser la fonction openOffMesh de TextureViewer.
- Charger un maillage, et l'afficher. (Vous pouvez utiliser la méthode drawMesh de TextureViewer)
- Calculer sa boîte englobante, envoyer les coordonnées minimums au GPU (ajouter des xMin, yMin, zMin) et mettre à jour votre normalisation. (Ou plus simple, redimensionner le maillage côté CPU pour qu'il occupe le même espace que l'image 3D)
- Qu'obtenez-vous comme résultat ?

Question 5 (bonus) : Rendu sur un ensemble de plans

- Pour rendre le volume, ne dessinez plus la boîte englobante, dessinez un ensemble de plans alignés sur les axes. Ajouter un discard des voxels dont la couleur est noir.
- Quel problème observez-vous ? Sur quelles directions doivent être alignés les plans pour palier ce problème ? Montrer le résultat.

Question 6 (bonus) : Rendu par raycasting

- Rendre la boîte englobante de l'image 3D,
- Dans le fragment shader, partir de la position P du Fragment pour lancer un rayon dans la grille dans la direction V (position de la camera) vers P.
- Utilisez du raymarching pour trouver la surface : parcourez la grille en avançant par petits pas le long du rayon en vous arrêtant lorsque la valeur de la texture dépasse 0. (Attention aux boucles infinies sur le GPU)
note : Si vous voulez rendre les plans de coupe fonctionnelles lors du raycasting, utilisez `computeVisibility` à chaque pas du rayon, et non plus sur les fragments.
- Pour calculer la normale au point d'intersection, calculer le gradient, sur les trois axes, de l'image 3D.